

VISOORA — AI SALES EMPLOYEE EXPANSION

Engineering Job Description & Scope of Work — Codex Implementation Brief

Adding Autonomous Email Outreach + Lead Research to the Existing AI Calling Platform, Without Breaking Verified Production-Hardening Work

Document Type: Ready-to-paste implementation brief for an AI coding agent (Codex / Claude Code / similar).

Audience: Autonomous coding agent with repo read/write access. No human design decisions should be required mid-task — this document is meant to be complete and self-contained.

Golden rule for the agent executing this: This is an ADDITION to an existing, partially-audited, partially-hardened codebase. Nothing described below may break, replace, mock, or regress any currently working call/telephony/FSM/VAD functionality, nor any of the security and data-integrity fixes already verified (auth, tenant isolation, analytics router, frontend real-data wiring). Treat all existing passing tests as a contract. If a new feature conflicts with an existing test, STOP and flag it — do not edit the test to make it pass.

1. Context — What Already Exists (Do Not Re-Break This)

Visoora is currently an AI outbound calling platform. A full technical audit and a subsequent production-hardening pass already addressed (or are in the process of addressing) the following — confirm current state in the repo before writing any new code:

- Real-time FSM/VAD telephony engine (Twilio + G.711 μ -law streaming) — STRONG, do not modify core logic.
- Auth bypass fix in `rbac.py` — gated to dev-only, ngrok bypass removed.
- Tenant isolation guard in `storage_manager.py::upload_recording` — `tenant_id` required, quarantine bucket for missing tenant.
- CSV lead import — now persists to local JSON when Supabase is offline (no silent data loss).
- Analytics router registered in `twilio_handler.py`; broken imports fixed.
- Frontend dashboard / campaigns / calls pages — being migrated off mock data ("Sarah Connor" hardcoded call) onto real `/api/logs`, `/api/campaigns`, `/analytics/*` calls.
- Live WebSocket wiring at `/api/live-ws` replacing the old `setInterval` mock loop.
- LLM fallback chain: Gemini → Gemini fallback model → GPT-4o → emergency scripted (last resort only).
- LLM guard fallback text fixed — no more "I am an AI assistant" persona breaks; uses `SAFE_RECOVERY_POOL` / `GROUNDING_RECOVERY`.
- TTS silence bug fixed — real ElevenLabs synthesis with Twilio `<Say>` as the genuine fallback, not silence.

Before writing a single line of new code, the agent must re-verify each item above against the CURRENT repo state (not this document, not prior chat history) — file paths, function names, and line numbers may have shifted.

2. Why This Expansion — Product Direction

Visoora is repositioning from "AI calling software" to an **AI Sales Employee**: an autonomous agent that is trained once on a business's sales knowledge, given a lead, researches that lead, decides the right channel and timing, and then acts — by phone and by email — continuously following up like a trained human SDR, not a fixed drip sequence.

This document scopes **Phase A** of that vision only: adding autonomous, non-templated email outreach and lead research on top of the existing calling engine. Multi-channel orchestration (LinkedIn, CRM hand-off, multi-agent Manager architecture) is explicitly OUT OF SCOPE for this phase — see Section 9.

3. Core Functional Requirement

Build the following end-to-end flow. Each numbered step below is a required capability, not a suggestion — all must be implemented for this phase to be considered complete.

3.1 — Create AI Sales Agent (Persistent, Project-Style)

Like a ChatGPT Project or Claude Project: the user creates one persistent Agent per use case/product line, not a one-off prompt per call.

- User provides: business description, product/service details, pricing, ICP, tone of voice, sales methodology, do's/don'ts.

- User uploads supporting documents: PDF, DOCX, plain text, and/or a website URL to crawl.
- All uploaded material is chunked, embedded, and stored in a per-agent knowledge store (vector DB) — this is the agent's persistent 'Sales Brain'. It must be re-usable across every call and email generated for that agent, not re-uploaded per lead.
- Agent configuration (persona, tone, pricing, objection-handling rules) must be stored as structured data AND be retrievable by both the existing call/FSM pipeline and the new email pipeline — both channels read from the SAME knowledge base and the SAME persona config. This is the most important architectural constraint in this entire document: do not build a second, divergent config system for email.

3.2 — Lead Intake

Reuse the existing lead/contact persistence path (CSV import + local JSON / Supabase fallback already fixed in the hardening pass) — extend its schema, do not replace it.

- Fields: name, company name, website, email, phone, LinkedIn (optional), free-text notes/description.
- Validate email format and phone format on intake; reject and report invalid rows rather than silently dropping them (same non-negotiable rule already applied to CSV import in the hardening pass).

3.3 — AI Research Step (Confirm Before Acting)

Before any outbound call or email is sent, the agent must research and confirm the lead — this is a hard gate, not optional enrichment.

- Fetch and parse the lead's website (company description, products, recent news if available).
- Cross-check the provided company name/website/email domain for basic consistency (e.g. email domain roughly matches company website domain) — flag mismatches rather than blocking, since data is often messy.
- Produce a structured 'lead brief': company summary, likely pain points, suggested personalization hooks, confidence score.
- If research cannot confirm the lead with reasonable confidence (e.g. dead website, bounced email pre-check), the agent must flag the lead as NEEDS_REVIEW and NOT proceed to outreach automatically. No silent best-effort guessing presented as fact.

3.4 — Outreach Strategy Decision (No Fixed Sequence)

This is the key product differentiator versus Instantly/Smartlead-style tools: there is no fixed Email-1 → wait → Email-2 → wait → Email-3 sequence. The agent decides the next action from current state.

- Inputs to the decision: lead brief, agent persona/knowledge base, channel history for this lead (was there a call? did they answer? did they open the email? did they reply?), time since last touch, configured calling-hour/TCPA compliance window.
- Possible decisions: call now, send email now, send email referencing a prior call, wait N hours/days, mark as no-response-stop, escalate to human, retry call.
- This decision logic must be implemented as an explicit, inspectable decision function/service (e.g. `OutreachStrategyEngine.decide_next_action(lead, history, agent_config)`) that returns a structured decision object with a reason string — NOT a black-box prompt with no logged reasoning. Every decision must be logged with the reason, so it can be audited and debugged.

3.5 — Email Action: Generation + Sending

- Email body is generated per-lead, per-touch, from the agent's knowledge base + lead brief + conversation/call history — never from a static template string. If a human could tell two emails were built from the same template with find/replace, this requirement has failed.
- Email generation and call-script generation should share the same underlying LLM prompting utility/persona context (re-use, do not fork, the persona-construction code already used for calls).
- Sending: integrate one transactional email provider (e.g. SendGrid, Postmark, or AWS SES — pick one and implement properly with bounce/complaint webhook handling; do not build a custom SMTP sender).
- Track: sent, delivered, opened, clicked, replied, bounced — store on the lead/touch record, used as input to Section 3.4's decision engine.
- Reply handling: an inbound parsing webhook/inbox must capture lead replies and feed them back into the lead's history so the next decision (call vs. wait vs. escalate) is reply-aware. Inbound replies must NOT auto-trigger another automated email without going through the same decision engine — no infinite bot-to-human-reply loops.

3.6 — Cross-Channel Memory

Every call and every email for a given lead must write to one shared interaction-history record. The call engine already logs to `local_call_logs.json` / Supabase `call_logs` — extend this schema (or add a clearly linked table) rather than creating a parallel, disconnected history for email. The dashboard, analytics, and the strategy decision engine must all read from this single source of truth.

4. Required Architecture Additions

4.1 — New Components

Component	Purpose	Integrates With
Agent Knowledge Store (vector DB)	Stores embedded chunks of uploaded docs/website pages	Call agent, LLM prompt builder, Email generator
Lead Research Service	Crawls website, builds lead brief, confidence score	Lead intake, Strategy engine
Outreach Strategy Engine	Decides next channel/action per lead per touch	Call trigger, Email trigger, Dashboard
Email Generation Service	LLM-generates non-templated email body per touch	Agent Knowledge Store, Lead history
Email Delivery Adapter	Sends via SendGrid/Postmark/SES, handles webhooks	Strategy engine, Interaction history
Unified Interaction History	Single timeline of calls + emails per lead	Dashboard, Analytics, Strategy engine

4.2 — Database / Schema Changes

- agents table/collection: id, tenant_id, name, persona_config (JSON), created_at, updated_at.
- agent_knowledge_chunks: agent_id, source_file, chunk_text, embedding_vector, created_at.
- leads: extend existing schema with research_brief (JSON), research_confidence, needs_review (bool).
- interaction_history: lead_id, tenant_id, channel ('call'|'email'), direction ('outbound'|'inbound'), content_ref, status, metadata (JSON), created_at — this REPLACES nothing; it is additive and must link to existing call_logs by foreign key/shared ID, not duplicate them.
- All new tables must include tenant_id as a required column from day one — the entire previous hardening milestone was about fixing a missing-tenant_id bug; do not reintroduce it in new tables.

4.3 — Non-Negotiable Constraints (Carried Over From Prior Hardening Work)

- Every new table/record must carry tenant_id. No 'default_tenant' shortcuts.
- Every new endpoint must go through the same auth dependency (Depends(get_current_user)) already standardized across the backend post-fix. No new auth bypass paths.
- No new mock data anywhere. If an external dependency (vector DB, email provider) isn't configured yet in a given environment, fail loudly with a clear error/log — do not silently fall back to fabricated success states.
- Local-fallback philosophy must be preserved: if Supabase/vector DB is offline in dev, follow the same local-JSON-fallback pattern already established for call logs and CSV leads — do not let the feature simply not work in local/dev mode.

5. Step-by-Step Implementation Guide for the Agent

Work in this exact order. Each step ends with a verification gate — do not proceed to the next step until the gate passes.

Step 1 — Audit current state

- 1 Re-run the full existing test suite. Record the baseline pass count (expected ~57+ passing).
- 2 Re-confirm the 9 items listed in Section 1 against current code (file/line). Note any drift from this document's assumptions.
- 3 GATE: Do not write new code until baseline is green and confirmed.

Step 2 — Agent + Knowledge Store

- 1 Create the agents and agent_knowledge_chunks schema (Section 4.2).
- 2 Build document ingestion: accept PDF/DOCX/plain text + website URL, chunk, embed, store.
- 3 Build a retrieval function used by BOTH the call FSM prompt builder and the new email generator — wire it into the existing call prompt construction code as a non-breaking addition (e.g. extra context injected, not a rewrite of the FSM itself).
- 4 GATE: Existing call tests still pass unmodified. New test: given an uploaded PDF, retrieval returns relevant chunks.

Step 3 — Lead Research Service

- 1 Build website crawl + summarization producing the structured lead brief described in Section 3.3.
- 2 Add research_brief, research_confidence, needs_review columns to leads.
- 3 GATE: Given a known test lead/website, the service returns a brief and a confidence score; a deliberately broken URL results in needs_review=true, not a crash or a fabricated brief.

Step 4 — Outreach Strategy Engine

- 1 Implement as an explicit, testable function per Section 3.4, returning a decision + logged reason.
- 2 Unit test every decision branch (no answer → voicemail → wait → email; email opened → call again; reply received → escalate/continue; no response after N touches → stop).
- 3 GATE: 100% branch coverage on the decision function with explicit test cases for each transition listed above.

Step 5 — Email Generation + Delivery

- 1 Integrate one provider (SendGrid/Postmark/SES) with API keys in env config, not hardcoded.
- 2 Build the LLM email generator sharing persona/knowledge context with the call pipeline (Section 3.5).
- 3 Implement delivery status + open/click/bounce webhooks writing into interaction_history.
- 4 Implement inbound reply capture writing into interaction_history and feeding the strategy engine.
- 5 GATE: Send a real test email end-to-end in a sandboxed/test inbox, confirm webhook events land in interaction_history, confirm a simulated reply changes the strategy engine's next decision.

Step 6 — Unified History + Dashboard Wiring

- 1 Add interaction_history writes alongside existing call_logs writes (additive, not replacing).
- 2 Extend the already-fixed dashboard/analytics endpoints to surface combined call+email activity — reuse the real-data wiring pattern already implemented in the hardening pass; do not reintroduce mock arrays.
- 3 GATE: A lead with 1 call + 2 emails shows a single combined timeline in the dashboard, sourced from real data.

Step 7 — End-to-end test

- 1 Run the full new-feature journey: create agent → upload knowledge docs → upload lead → research runs → strategy decides → call OR email fires → follow-up logic reacts to outcome → dashboard reflects it.
- 2 Re-run the FULL original test suite (calling/FSM/auth/tenant/analytics) to confirm zero regressions.
- 3 GATE: Both the new journey and 100% of pre-existing tests pass before this phase is marked done.

6. Rules the Coding Agent Must Follow at All Times

- Never delete or rewrite the existing FSM, VAD, or telephony streaming code to 'make room' for the email feature. It is additive only.
- Never edit an existing passing test to force it to pass after a change — if a change breaks an existing test, the change is wrong, not the test.
- Never introduce a second, parallel config/persona system for email that diverges from the one used for calls. One agent, one persona, one knowledge base, two channels.
- Never silently swallow errors from the email provider, vector DB, or research crawler. Log clearly, surface clearly, and where appropriate set `needs_review` rather than guessing.
- Never hardcode `tenant_id`, API keys, or provider credentials in source — use environment configuration consistent with the existing project conventions.
- Never auto-send more than one outbound touch (call or email) without the Strategy Engine explicitly approving the next one — no bypassing the decision gate for 'efficiency'.
- Always run the full existing test suite before AND after each step in Section 5, and report both results.

7. Definition of Done

This phase is complete only when ALL of the following are true:

- A user can create an Agent, upload documents/website, and the agent's responses (in both call and email) demonstrably reflect that uploaded content (not generic boilerplate).
- A user can upload a lead and the system autonomously researches it, decides a channel, and acts — without a fixed template sequence.
- Two emails generated for two different leads from the same agent are demonstrably different in wording/personalization, not just name/company swapped into an identical template.
- Call and email history for a lead appear together in one timeline in the dashboard.
- All pre-existing tests (calling, auth, tenant isolation, analytics, CSV import) still pass with zero regressions.
- No new mock data, no new auth bypass, no `tenant_id` gaps anywhere in the new code paths.

8. Required Reporting Format From the Agent After Each Step

- Step name and goal.
- Files created/changed.
- Tests added and their pass/fail result (show actual output, not a summary claim).
- Confirmation that the full pre-existing test suite was re-run and its result.
- Any deviation from this document, with reasoning, flagged explicitly — do not silently deviate.

9. Explicitly Out of Scope for This Phase

- LinkedIn messaging/automation.
- Multi-agent Manager/Research/Qualification/CRM-agent architecture — this phase uses one Strategy Engine function, not a multi-agent system. Multi-agent orchestration is a future phase, not part of this build.
- Full CRM (HubSpot/Salesforce) two-way sync — only internal `interaction_history` is required now.

- Voicemail-drop audio generation beyond what already exists in the call engine.
- Enterprise approval workflows / human-in-the-loop UI beyond the simple needs_review flag.

10. One-Sentence Summary for the Agent

Add a persistent, document-trained Sales Agent and a non-templated email channel that shares the same knowledge base and decision history as the existing call engine, governed by one auditable Strategy Engine, without breaking, mocking, or regressing any existing tested functionality.